AFDB · DATA INNOVATION FOR AFRICA (DI4A) · HANDS-ON SERIES

Engines of Scale

Searching, aggregating and mapping big data with the Elastic Stack (ELK)

From Ookla Speedtest tiles to a population-weighted, subnational connectivity indicator

Big data tools for official statistics — Innovating the Data Value Chain
African Development Bank – Data Innovation for Africa (DI4A)

SECTION 01

01

Why scale matters

When does a laptop and pandas stop being enough?



The big data value chain: six building blocks

IN FOCUS

01

Ingestion

Collect files, APIs and feeds; validate on arrival.

S3 downloads, APIs, Logstash

02

Storage

Data lake for raw files; lakehouse for governed tables.

Parquet, Delta, Iceberg

03

Processing

Batch or streaming transformation, spread across machines.

Spark, DuckDB, pandas

04

Indexing & search

Pre-computed structures that answer filters in milliseconds.

Elasticsearch, OpenSearch

CROSS-CUTTING

Orchestration — Schedules, retries and monitoring — the quarterly Ookla refresh runs itself (e.g. Airflow, cron).

Metadata & versioning — Which file, which code, which mapping produced this figure? Catalogues, Git and data versioning.

Lake, lakehouse or search index: which does what?

Data lake

Answers: Where is the raw file?

- Files on object storage (Parquet, CSV, rasters)
- Schema-on-read, very low cost
- Keeps the untouched source for audit

Lakehouse

Answers: What is the official aggregate?

- Tables on the lake (Delta Lake, Apache Iceberg)
- SQL, transactions and time travel
- Reproducible production of statistics

Search index

Answers: What does this slice look like, now?

- Elasticsearch / OpenSearch
- Inverted index and BKD trees
- Sub-second filters, text, geo, dashboards

Batch

Process a bounded dataset at intervals — e.g. each quarterly Ookla release.

Streaming

Process unbounded events as they arrive — e.g. sensors, network or transaction feeds.

The Ookla case: why this dataset tests your tools

610.8 m
tile edge at the equator (web-Mercator zoom level 16)

Q1 2019
first quarter available; new tiles every quarter since

2 series
fixed broadband and mobile, published separately

KEY FIELDS IN EACH TILE

`quadkey` `avg_d_kbps` `avg_u_kbps` `avg_lat_ms` `tests` `devices` `tile (WKT)`

Why a single laptop struggles. Ookla aggregates hundreds of millions of monthly tests into tiles, yet a multi-country, multi-quarter, fixed + mobile panel still grows fast. Add geometry, joins to boundaries and WorldPop, and several colleagues asking questions at the same time — and memory, not skill, becomes the bottleneck.

Source: Ookla, ookla-open-data README (GitHub) and Registry of Open Data on AWS — licence CC BY-NC-SA 4.0.

SECTION 02

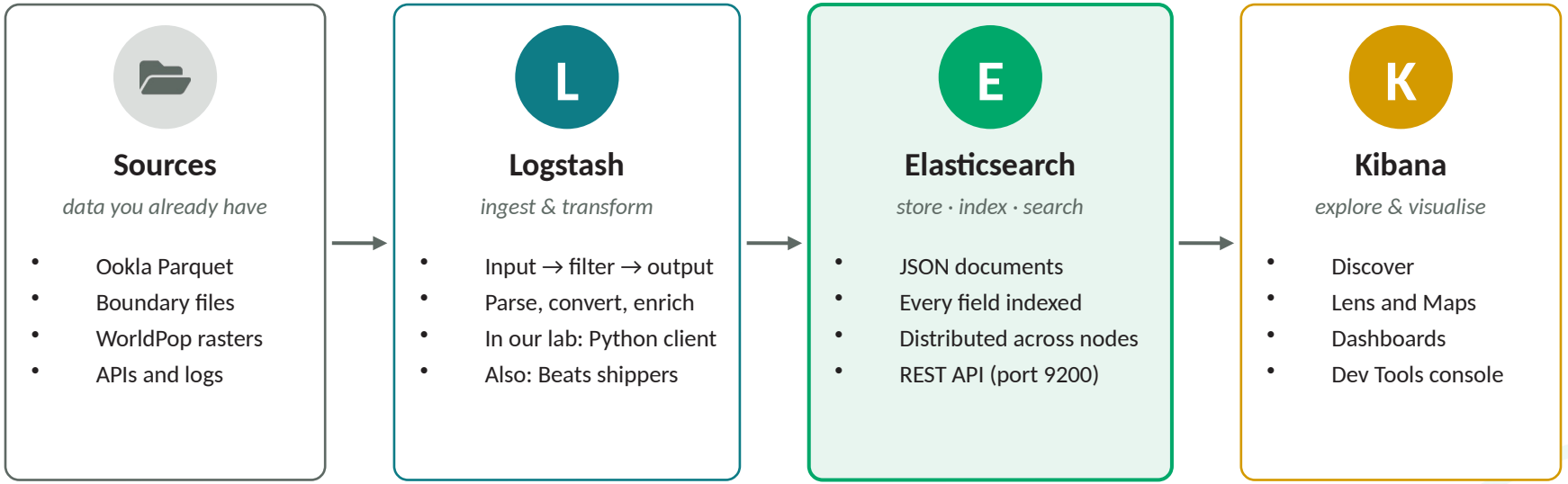
02

The Elastic Stack explained

What exactly are E, L and K — and how do they work together?



ELK in one picture



From "ELK" to "Elastic Stack". Beats (lightweight data shippers) joined E, L and K, so Elastic renamed the family. People still say "ELK" — both names refer to the same toolset.

The core idea: the inverted index

DOCUMENTS

- **D1** Kigali · urban · mobile
- **D2** Musanze · rural · fixed
- **D3** Kigali · rural · mobile

index once →

TERM → DOCUMENTS

`kigali`	D1, D3
`musanze`	D2
`urban`	D1
`rural`	D2, D3
`mobile`	D1, D3
`fixed`	D2

QUERY

`kigali AND rural`

{D1, D3} ∩ {D2, D3}
= {D3}

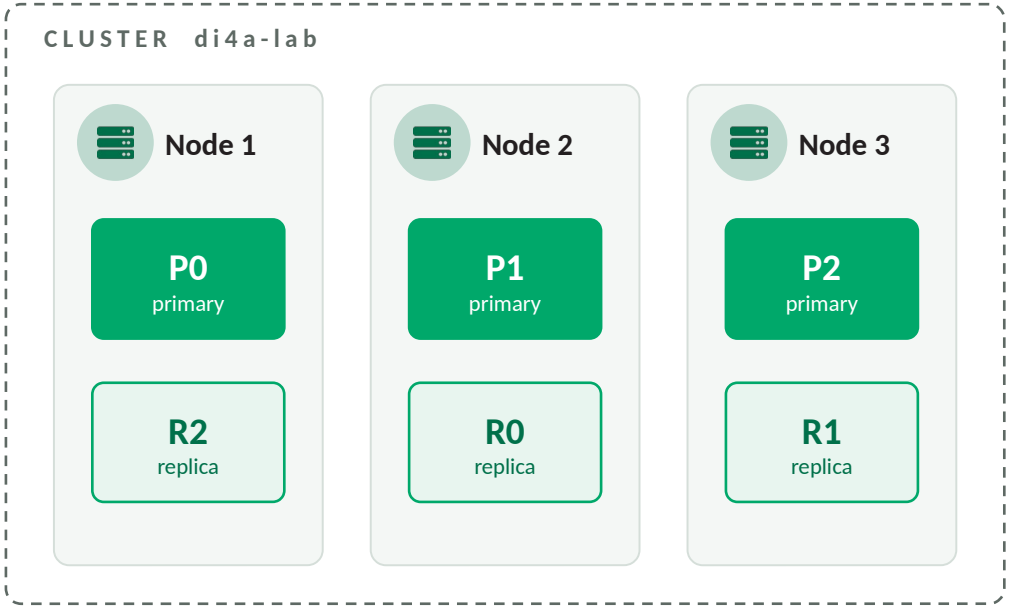
Found without reading a single document — only two short lists are compared.

Think of the index at the back of a statistical yearbook: look up the term, jump straight to the pages. Numbers and coordinates use a sibling structure (BKD trees) for ranges and geography. The trade-off: indexing costs time, disk and memory up front, so queries can be fast later.

Speaking Elasticsearch: a translation guide from pandas and SQL

IN PANDAS / SQL		IN ELASTICSEARCH	WHAT IT MEANS IN PRACTICE
`DataFrame / table`	→	**Index**	A named collection, e.g. ookla-tiles
`Row`	→	**Document**	One tile per quarter, stored as JSON with a unique _id
`Column`	→	**Field**	quadkey, avg_d_kbps, tile … each is indexed
`dtypes / schema`	→	**Mapping**	Declared types; changing one later means reindexing
`df[filter] / WHERE`	→	**Query**	Query DSL (JSON) or ES|QL (piped text)
`groupby().agg() / GROUP BY`	→	**Aggregation**	Buckets (groups) and metrics (statistics)

Anatomy of a cluster: nodes, shards and replicas



ookla-tiles: 3 primaries (P) × 1 replica (R) — never on the same node.

Node

One running Elasticsearch process, usually one machine.

Shard

A slice of an index; the unit that moves between nodes.

Replica

A copy on another node: resilience plus extra search capacity.

SIZING GUIDANCE

10–50 GB

per shard, with fewer than 200 million documents each. A national Ookla index needs only one or two primaries.

Source: Elastic documentation, "Size your shards".

SECTION 03

03

Modelling statistical data

How do we turn an Ookla tile into a searchable document?



From a Parquet row to an enriched JSON document

AS PUBLISHED BY OOKLA

```
quadkey    "3001010303300022"
avg_d_kbps 21450
avg_u_kbps 9870
avg_lat_ms 38
tests      142
devices    57
tile       "POLYGON((30.0586
           -1.9552, ...))"
```

ENRICH

INDEXED IN ELASTICSEARCH

```
{
  "quadkey": "3001010303300022",
  "service_type": "mobile",
  "quarter": "2025-Q2",
  "quarter_start": "2025-04-01",
  "avg_d_kbps": 21450, "avg_u_kbps": 9870,
  "avg_lat_ms": 38, "tests": 142, "devices": 57,
  "admin1": "Kigali City",
  "admin2": "Nyarugenge",
  "pop_worldpop": 4830.5,
  "centroid": {"lat": -1.9525, "lon": 30.0614},
  "tile": "POLYGON((30.0586 -1.9552, ...))"
}
```

- **Quarter & service** from the file path
- **admin1 / admin2** spatial join with boundaries
- **pop_worldpop** zonal sum from the previous lab
- **centroid** computed from the polygon

Green text = added during enrichment

Quadkey and polygon are a real zoom-16 tile in Kigali; speeds, tests and population are illustrative values.

Mapping: declare the schema before you index

```
PUT ookla-tiles
{
  "mappings": {
    "dynamic": "strict",   ◀ reject surprises
    "properties": {
      "quadkey":       {"type": "keyword"},
      "service_type":  {"type": "keyword"},
      "quarter":       {"type": "keyword"},
      "quarter_start": {"type": "date"},
      "avg_d_kbps":    {"type": "integer"},
      "avg_u_kbps":    {"type": "integer"},
      "avg_lat_ms":    {"type": "integer"},
      "tests":         {"type": "integer"},
      "devices":       {"type": "integer"},
      "admin1":        {"type": "keyword"},
      "admin2":        {"type": "keyword"},
      "pop_worldpop":  {"type": "float"},
      "centroid":      {"type": "geo_point"},
      "tile":          {"type": "geo_shape"}
}}}
```

keyword — Exact values: codes, names, categories. Filters, grouping, sorting.

integer · float — Measurements and counts. Ranges and statistics.

date — Time filters and date histograms (quarter_start).

geo_point · geo_shape — Coordinates and polygons for spatial queries and maps.



Keep codes as text. A tile over Dakar has quadkey 0333…; cast to a number, it silently loses its leading zero. Map quadkeys and admin codes as keyword.

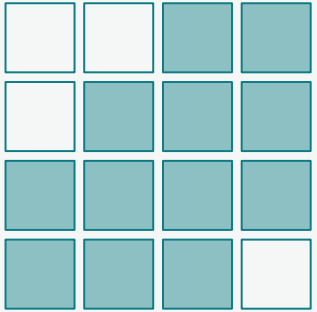
geo_point or geo_shape? Two views of the same tile



geo_point

the tile centroid

- Compact and very fast
- Heatmaps and geotile_grid buckets
- Distance queries ("within 10 km")
- A border tile belongs to one side only



geo_shape

the full 610 m polygon

- Exact intersection with boundaries
- Border tiles counted by both regions
- Accurate polygon rendering in Maps
- Heavier to index and to query

Recommendation: store both. Centroid for speed, polygon for exact boundary intersections. Mind the axis order: WKT and GeoJSON are longitude first, as in the Ookla files.

Four ways to get data in — and the one we use in the lab

LAB 1

Python client

Bulk helper in a notebook. Scriptable, versioned, reproducible.

Kibana file upload

Drag-and-drop a small CSV or NDJSON file to explore quickly.

Logstash

Recurring pipelines: input, parse, enrich, output.

Ingest pipelines

Processing inside Elasticsearch: convert, rename, enrich.

```
import pandas as pd
from elasticsearch import Elasticsearch
from elasticsearch import helpers

es = Elasticsearch(URL, api_key=KEY)
df = pd.read_parquet("rw_tiles.parquet")

def actions(df):
    for r in df.to_dict("records"):
        yield {
          "_index": "ookla-tiles",
          # same tile = same id: no duplicates
          "_id": f"{r['quadkey']}-"
                 f"{r['quarter']}-mobile",
          "_source": r}

ok, errors = helpers.bulk(
    es, actions(df), chunk_size=5000)
```

SECTION 04

04

Querying and aggregating

How do we ask the cluster statistical questions?



Anatomy of a search request

```
GET ookla-tiles/_search
{
  "size": 0,  ◀ 1
  "query": {
    "bool": {
      "filter": [  ◀ 2
        {"term": {"service_type": "mobile"}},
        {"term": {"quarter": "2025-Q2"}}
      ]
    }
  },
  "aggs": {
    "by_region": {
      "terms": {"field": "admin1", "size": 50},  ◀ 3
      "aggs": {
        "mean_dl": {
          "weighted_avg": {  ◀ 4
            "value":  {"field": "avg_d_kbps"},
            "weight": {"field": "tests"}
}}}}}
```

1

`size: 0`

Return only the aggregated results — not the thousands of documents behind them.

2

`filter`

Yes/no conditions: no relevance scoring, cacheable, fast. This is your WHERE clause.

3

`terms`

One bucket per region — the GROUP BY. Set size above your number of regions.

4

`weighted_avg`

Mean download speed per region, weighted by the number of tests in each tile.

The aggregation toolbox — and its statistical fine print

Metric
compute a value

avg · sum · min · max
≈ mean(), sum()

weighted_avg
≈ np.average(w=)

percentiles
≈ quantile()

cardinality
≈ nunique()

Bucket
build groups

terms
≈ groupby(col)

range · histogram
≈ pd.cut()

date_histogram
≈ resample()

geotile_grid
≈ group by map tile

Pipeline
work on other aggs

derivative
≈ diff()

cumulative_sum
≈ cumsum()

bucket_sort
≈ sort + head()

bucket_script
≈ computed ratio

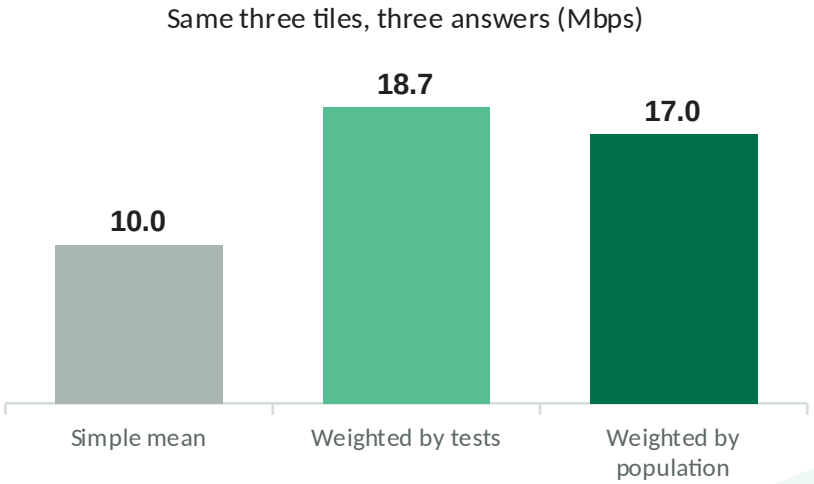
Approximate by design: cardinality (HyperLogLog++), percentiles (TDigest) and multi-shard terms counts. Fine for exploration — verify precision or compute exactly before publishing.

Building a population-weighted connectivity indicator

FORMULA FOR REGION r

$$c_r = \Sigma\,(pop_i \times speed_i) \div \Sigma\, pop_i$$

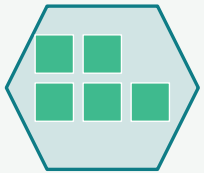
Tile	Population	Tests	Mbps
A · urban	5,000	400	**20**
B · peri-urban	1,200	40	**8**
C · rural	300	5	**2**



Two rules. Document which weight you used — it changes the result. And tiles without tests have no speed, so their population drops out: publish the share of population covered next to the indicator.

Illustrative figures for teaching purposes, not observed data. In Elasticsearch: weighted_avg with weight field pop_worldpop.

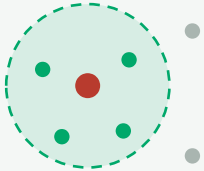
Asking geographic questions



geo_shape

Which tiles intersect this district?

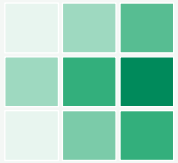
Subnational indicators, border tiles included.



geo_distance

Which tiles lie within 10 km of this point?

Access around hospitals, schools or markets.



geotile_grid

How many tests per map cell?

Heatmaps and multi-scale summaries.

REUSE A STORED BOUNDARY INSTEAD OF PASTING COORDINATES

```
"query": {"bool": {"filter": {"geo_shape": {"tile": {
  "indexed_shape": {"index": "admin-boundaries", "id": "RW-02", "path": "geometry"},
  "relation": "intersects"}}}}}
```

ES|QL: the same question, written as a pipeline

```
FROM ookla-tiles
| WHERE service_type == "mobile"
    AND quarter == "2025-Q2"
| EVAL w = TO_DOUBLE(avg_d_kbps) * tests   ◀ use decimals
| STATS sum_w = SUM(w),
        n_tests = SUM(tests) BY admin1
| EVAL mean_mbps = ROUND(sum_w / n_tests / 1000, 1)
| SORT mean_mbps DESC
| KEEP admin1, mean_mbps, n_tests
```



Each step receives the table produced by the previous one — like a pipe in R or a chain in pandas.

Query DSL · *JSON*

- Every Elasticsearch feature
- Best for applications and dashboards
- Verbose, but precise

ES|QL · *piped text*

- Reads like a recipe
- Ideal for exploration in Discover
- GA since 8.14 (June 2024)

Source: Elastic, "Elastic 8.14: Announcing the GA of ES|QL" (June 2024).

SECTION 05

05

Exploring with Kibana

How do colleagues who do not code get answers from the cluster?



Kibana: five tools you will use in the lab

Dev Tools

Console to run Query DSL and ES|QL requests.

Lab 1: mapping, indexing checks, first queries

Discover

Browse and filter documents; inspect fields.

Lab 2: sanity checks, KQL filters

Lens

Drag-and-drop charts: bars, lines, tables.

Lab 2: speed by quarter, fixed vs mobile

Maps

Tile polygons, points and choropleths.

Lab 2: region map, tile zoom-in

Dashboards

Combine panels with shared filters.

Lab 2: the connectivity explorer

KQL — THE FILTER BAR LANGUAGE IN DISCOVER AND DASHBOARDS

```
service_type : "mobile" and tests >= 5 and admin1 : "Kigali City"
```

Lab 2 workflow: building a search-driven connectivity explorer

1	2	3	4	5	6
Data view	**Discover**	**Map**	**Lens**	**Dashboard**	**Benchmark & publish**
ookla-tiles, time field quarter_start	Check fields, drop tiles with few tests	Choropleth of admin1 by weighted speed	Speed by quarter, fixed vs mobile	Add region and quarter controls	Compare with pandas; commit to GitHub
5 min	5 min	15 min	10 min	10 min	15 min

Deliverable: *a dashboard that answers "Where is mobile connectivity weakest, and is it improving?"* — plus a screenshot and the saved-objects export (NDJSON) committed to your team repository.

pandas or Elasticsearch? Decide with your own evidence

Stay with pandas / DuckDB when...

- The data fits comfortably in memory
- One analyst, one-off or periodic analysis
- The reproducible script is the product
- No server skills available to maintain a cluster

Move to Elasticsearch when...

- Data outgrows a single machine
- Many colleagues need interactive answers
- Repeated filtered and spatial queries
- New data arrives continuously

YOUR BENCHMARK — FILL IN DURING LAB 2

Task (same data, same question)	pandas time	pandas memory	ES query time	ES disk / heap
1 · Load or index one quarter	…	…	…	…
2 · Weighted mean speed by region	…	…	…	…
3 · Tiles intersecting one district	…	…	…	…

SECTION 06

06

Governing and sustaining

What must be true before an NSO publishes from a search engine?



Quality and ethics: what the index does not fix

Coverage bias

Crowdsourced, self-selected testers — not a probability sample. Expect urban and smartphone skew.

Small numbers

Set a minimum-tests threshold; suppress or flag regions below it and always report n.

Licence

CC BY-NC-SA 4.0: attribute Ookla, non-commercial use, share derived data alike.

Confidentiality

No microdata on open clusters. Authentication, role-based access, encrypted transport.

Reproducibility

Mapping, ingestion code and queries versioned in Git with the data vintage.

Honest limits

A proxy is not an official statistic. Say what it cannot support, in writing.

Licence: Registry of Open Data on AWS, Speedtest by Ookla Global Fixed and Mobile Network Performance Maps.

Deployment options for a statistical office

Self-managed

on premises or national data centre

- \+ Full control of data location
- \+ No per-use cloud bill
- – Needs skills: upgrades, security, backups

Managed cloud

Elastic Cloud or another provider

- \+ Running in minutes, scales easily
- \+ Operations handled by the provider
- – Data residency and recurring cost

OpenSearch

Apache 2.0 fork, Linux Foundation

- \+ Fully open-source licence
- \+ Concepts from this session transfer
- – APIs have diverged in places

Licensing in brief. Since September 2024, the Elasticsearch and Kibana source code can be used under the OSI-approved AGPLv3, alongside SSPL and the Elastic License; default downloads stay under the Elastic License. Confirm the terms with your legal team before production use.

Sources: Elastic, "FAQ on Software Licensing" and "Elasticsearch is Open Source. Again!"; Revenera (2025) on OpenSearch.

Publishing on GitHub: the repository your team leaves with

SUGGESTED STRUCTURE

```
di4a-<country>-connectivity/
├── README.md
├── LICENSE
├── mappings/
│   └── ookla-tiles.json
├── notebooks/
│   ├── 01_index_tiles.ipynb
│   └── 02_benchmark.ipynb
├── queries/
│   ├── by_region.json
│   └── by_region.esql
├── kibana/
│   └── dashboard.ndjson
└── outputs/
    └── connectivity_by_region.csv
```

THE README MUST STATE

- **Method**
  Enrichment, weighting choice, minimum-tests threshold.
- **Sources**
  Ookla, WorldPop and boundaries — with vintage and access date.
- **Licences**
  CC BY-NC-SA 4.0 for derived data; a separate licence for code.
- **Limitations**
  Coverage bias, population not covered, what the proxy cannot support.
- **How to reproduce**
  Software versions, commands, expected outputs.

Feeds → National connectivity statistics · country presentations

KEY TAKEAWAYS

Six ideas to take back to your office

01 **Search is one block of the chain**

Keep raw files in the lake; index a curated, enriched subset.

02 **The inverted index is the secret**

Work done at indexing time buys speed at query time.

03 **Model before you load**

Strict mappings, codes as keyword, both point and shape.

04 **Weights are a methodological choice**

Document them, and publish population coverage.

05 **Kibana opens data to non-coders**

But decide on tooling with your own benchmark.

06 **Speed never replaces quality**

Licence, confidentiality and honest limitations come first.

Sources

Series	African Development Bank – Data Innovation for Africa (DI4A): hands-on series on big data tools for official statistics.
Ookla data	Ookla — ookla-open-data README, github.com/teamookla/ookla-open-data (tile size, fields, quarterly releases since Q1 2019).
Ookla licence	Registry of Open Data on AWS — Speedtest by Ookla Global Fixed and Mobile Network Performance Maps (CC BY-NC-SA 4.0).
Shard sizing	Elastic documentation — "Size your shards", elastic.co/docs (10–50 GB per shard, under 200 million documents).
ES|QL	Elastic blog — "Elastic 8.14: Announcing the GA of Elasticsearch Query Language (ES|QL)", June 2024.
Licensing	Elastic — "FAQ on Software Licensing" and S. Banon, "Elasticsearch is Open Source. Again!" (AGPLv3 option, 2024).
OpenSearch	Revenera blog — "Elastic's Return to Open Source" (January 2025): OpenSearch fork and transfer to the Linux Foundation.

QUESTIONS · THEN TO THE LAB

From tiles to insight — at scale

Your cluster is running. Your notebooks are waiting. Sit with your team.